

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 9: Practical Exercises

Foreign Trade University

Instructions

For each problem below there is a starter file `exercises/starter_code_problemNN.py` containing function signatures with `raise NotImplementedError` bodies, and a small `__main__` block with tests. Implement the missing functions so the tests pass. A reference solution is provided in `solutions/solution_problemNN.py` – try the problem yourself before looking!

All problems use only the Python 3.10+ standard library. Shared helper functions live in `starter_code.py` at the week root: `random_string`, `generate_weighted_graph`, `generate_dag`, `adjacency_list`, `adjacency_matrix`, `reconstruct_path`, `grid_from_seed`, and the constant `INF`.

The 12 problems are grouped into three themes:

- **Sequence Alignment** (Problems 1–5): edit distance, alignment with custom costs, reconstruction, longest common subsequence, and the LCS–edit-distance identity.
- **Shortest Paths** (Problems 6–10): Bellman–Ford distances and negative-cycle detection, Floyd–Warshall all-pairs distances, cross-verification, and DAG shortest paths via topological order.
- **More DP** (Problems 11–12): matrix chain multiplication with optimal parenthesisation, and grid path counting / minimum-path-sum with reconstruction.

1 Sequence Alignment

Problem 1 – Edit Distance (Levenshtein)

Implement `edit_distance(x, y)` returning the Levenshtein edit distance between two strings: the minimum number of single-character *insertions*, *deletions*, and *substitutions* that turn `x` into `y`. Use the standard 2-D DP with recurrence

$$\text{OPT}(i, j) = \min([x_i \neq y_j] + \text{OPT}(i-1, j-1), 1 + \text{OPT}(i-1, j), 1 + \text{OPT}(i, j-1)),$$

$$\text{OPT}(i, 0) = i, \text{OPT}(0, j) = j.$$

Example. `edit_distance("kitten", "sitting") == 3`.

Problem 2 – Sequence Alignment with Custom Gap and Mismatch Costs

Implement `alignment_cost(x, y, gap, mismatch)` returning the minimum total cost of aligning `x` and `y`, where each gap (insertion or deletion) costs `gap` and each mismatch (substituting one character for a different one) costs `mismatch`; matching equal characters costs 0. This generalises Problem 1 (which is the case `gap == mismatch == 1`).

Example. With `gap=2, mismatch=3`, aligning "AT" and "AAT" costs 2 (insert one A; the rest matches).

Problem 3 – Reconstruct an Optimal Alignment

Implement `align(x, y, gap, mismatch)` returning `(cost, ax, ay)` where `ax` and `ay` are the two input strings padded with the gap symbol '-' so they have equal length and represent an optimal alignment (same cost as Problem 2). Fill the DP table, then *trace back* from (m, n) to $(0, 0)$, emitting one column per step.

Example. `align("AT", "AAT", gap=1, mismatch=1)` can return `(1, "-AT", "AAT")` (costs may tie; any optimal alignment of the right cost is accepted by the tests, which re-check the cost via Problem 2).

Problem 4 – Longest Common Subsequence (Length + One LCS)

Implement `lcs(x, y)` returning `(length, subseq)` where `length` is the length of a longest common subsequence of `x` and `y`, and `subseq` is one such subsequence (a string). Use the standard DP and trace back to build the subsequence.

Example. `lcs("AGGTAB", "GXTXAYB")` returns `(4, "GTAB")`.

Problem 5 – Verify the LCS / Edit-Distance Relation

The insert/delete-only edit distance (no substitutions) satisfies $d(x, y) = |x| + |y| - 2L$, where L is the LCS length. Implement `indel_distance(x, y)` computing the minimum number of insertions and deletions to turn `x` into `y` *directly* via DP (substitution forbidden – i.e. a mismatch column is not allowed; only matches and gaps). Then implement `verify_lcs_relation(num_trials, length, seed)` that, for `num_trials` random string pairs (via `random_string`), checks `indel_distance(x, y) == len(x) + len(y) - 2 * lcs(x, y)[0]` for every pair, returning `True` iff the identity always holds.

2 Shortest Paths

Problem 6 – Bellman–Ford Shortest-Path Distances

Implement `bellman_ford(n, edges, source)` returning a list `dist` of length `n`, where `dist[v]` is the shortest-path distance from `source` to `v` (using `INF` for unreachable vertices). The graph `(n, edges)` may have *negative* edge weights but is assumed to have no negative cycle reachable from the source. Relax all edges `n - 1` times.

Example. For `n=3`, `edges=[(0,1,4),(0,2,5),(1,2,-3)]`, `source=0`, the distances are `[0, 4, 1]` (the path $0 \rightarrow 1 \rightarrow 2$ costs $4 + (-3) = 1$).

Problem 7 – Bellman–Ford Negative-Cycle Detection

Implement `find_negative_cycle(n, edges)` returning `None` if the graph has no negative cycle, or otherwise a list of vertices `[c0, c1, ..., c0]` describing one negative cycle (start and end vertex repeated). Run Bellman–Ford-style relaxation (initialising all distances to 0 so that *every* vertex is a potential source), then do one extra pass: if some edge still relaxes, follow parent pointers `n` times to land inside the cycle, then walk it out.

Example. `find_negative_cycle(2, [(0,1,1),(1,0,-3)])` returns a cycle of total weight $1 + (-3) = -2$, e.g. `[0, 1, 0]` (or `[1, 0, 1]`). The tests check that the returned cycle is genuine and has negative total weight.

Problem 8 – Floyd–Warshall All-Pairs Shortest Paths

Implement `floyd_warshall(n, edges)` returning an `n`-by-`n` matrix `dist` where `dist[i][j]` is the shortest-path distance from `i` to `j` (`INF` if no path; 0 on the diagonal). Assume no negative cycle. Use the classic triple loop over `k, i, j`.

Example. For `n=3`, `edges=[(0,1,4),(0,2,5),(1,2,-3)]`, `dist[0]` is `[0, 4, 1]`.

Problem 9 – Verify Bellman–Ford Agrees with Floyd–Warshall

Implement `verify_bf_vs_fw(num_trials, n, m, seed)`: for each of `num_trials` random graphs (via `generate_weighted_graph(n, m, seed=seed + trial)`) that contain *no* negative cycle (skip / regenerate those that do, using Problem 7), check that for source 0, `bellman_ford(n, edges, 0)` equals row 0 of `floyd_warshall(n, edges)`. Return `True` iff they always agree.

Problem 10 – DAG Shortest Path via Topological Order

Implement `dag_shortest_path(n, edges, source)` for a directed *acyclic* graph: compute a topological order (e.g. Kahn’s algorithm), then relax edges once in that order, returning the `dist` list. Then implement `verify_dag_matches_bellman_ford(num_trials, n, m, seed)` that, on random DAGs (via `generate_dag`), checks that the linear-time DAG sweep produces the same distances as the general Bellman–Ford – confirming the simpler algorithm suffices on a DAG.

3 More DP

Problem 11 – Matrix Chain Multiplication

Given dimensions `dims` (a list of length $n+1$, so matrix A_i is `dims[i-1]` by `dims[i]` for $i = 1, \dots, n$), implement `matrix_chain(dims)` returning (`min_cost`, `parenthesization`) where `min_cost` is the minimum number of scalar multiplications and `parenthesization` is a string such as "`((A1A2)A3)`" describing one optimal order. Use the interval DP

$$M(i, j) = \min_{i \leq k < j} (M(i, k) + M(k + 1, j) + p_{i-1}p_kp_j),$$

storing the optimal split point to reconstruct the parenthesisation.

Example. `matrix_chain([10, 30, 5, 60])` returns (4500, "`((A1A2)A3)`") ($10 \cdot 30 \cdot 5 + 10 \cdot 5 \cdot 60 = 1500 + 3000 = 4500$, beating the alternative $30 \cdot 5 \cdot 60 + 10 \cdot 30 \cdot 60 = 27000$).

Problem 12 – Grid Unique Paths and Minimum Path Sum

For an r -by- c grid with movement restricted to *right* and *down*:

- Implement `unique_paths(r, c)` returning the number of distinct paths from the top-left to the bottom-right corner ($P(i, j) = P(i - 1, j) + P(i, j - 1)$).
- Implement `min_path_sum(grid)` returning (`cost`, `path`) where `grid` is a list of lists of cell costs, `cost` is the minimum total cost of a top-left-to-bottom-right path, and `path` is one optimal path as a list of (`row`, `col`) coordinates (reconstructed by `traceback`).

Example. `unique_paths(3, 3) == 6`. `min_path_sum([[1,3,1],[1,5,1],[4,2,1]])` returns cost 7 with path `[(0,0),(0,1),(0,2),(1,2),(2,2)]` (down the costs 1,3,1,1,1).

4 LeetCode Practice

After completing the problems above, practise this week's dynamic-programming techniques (sequence alignment, LCS, grid DP, and shortest-path DP) on the following [LeetCode](#) problems. Difficulty is LeetCode's own label.

- [Longest Common Subsequence](#) – *Medium* – 2-D DP.
- [Edit Distance](#) – *Medium* – sequence alignment.
- [Delete Operation for Two Strings](#) – *Medium* – LCS variant.
- [Minimum ASCII Delete Sum for Two Strings](#) – *Medium* – sequence alignment.
- [Distinct Subsequences](#) – *Hard* – 2-D DP.
- [Longest Increasing Subsequence](#) – *Medium* – DP / patience sorting.
- [Unique Paths](#) – *Medium* – grid DP.
- [Minimum Path Sum](#) – *Medium* – grid DP.
- [Regular Expression Matching](#) – *Hard* – 2-D DP.
- [Cheapest Flights Within K Stops](#) – *Medium* – Bellman–Ford DP.